

Análise e Correção do Código de Treliza Plana

Este documento apresenta a análise do código original enviado para resolução de treliças planas, as correções realizadas e uma versão reestruturada e mais robusta do programa.

Critério	Avaliação
Execução	O código original executava após correção do <code>__main__</code> .
Cálculos	Foi corrigido o cálculo do comprimento das barras.
Método Numérico	Mantido Gauss manual, com proteção contra pivô nulo.
Gráficos	Melhorados títulos, escalas e visualização.
Estrutura	Código reorganizado em funções e etapas mais claras.

Principais correções realizadas:

- Correção do cálculo do comprimento da barra: $dx^{**2} + dy^{**2}$.
- Correção do bloco principal: `__name__ == "__main__"`.
- Inclusão de verificação de pivô nulo no método de Gauss.
- Separação mais clara entre montagem da matriz, resolução e gráficos.
- Inclusão de títulos e ajustes gráficos para melhor interpretação.
- Melhor tratamento visual para tração e compressão.

Código Refeito e Corrigido

```
import numpy as np
import matplotlib.pyplot as plt

def gauss_elimination(A, b):
    n = len(b)
    M = np.hstack([A.astype(float), b.reshape(-1,1)])

    for i in range(n):

        # Pivotamento parcial
        max_row = i + np.argmax(np.abs(M[i:, i]))

        if abs(M[max_row, i]) < 1e-12:
            raise ValueError("Sistema singular ou pivô nulo.")

        M[[i, max_row]] = M[[max_row, i]]

        # Eliminação
        for k in range(i+1, n):
            fator = M[k, i] / M[i, i]
            M[k, i:] -= fator * M[i, i:]

        # Substituição regressiva
        x = np.zeros(n)

        for i in range(n-1, -1, -1):
            x[i] = (M[i, -1] - np.dot(M[i, i+1:n], x[i+1:n])) / M[i, i]

    return x

def resolver_treliza():
    nos = np.array([
```

```

[0.0, 0.0],
[2.0, 0.0],
[1.0, 1.732],
[1.0, 0.0]
])

barras = np.array([
[0, 3],
[3, 1],
[0, 2],
[2, 1],
[2, 3]
])

N = len(nos)
B = len(barras)

F = np.zeros(2*N)
F[2*3 + 1] = -1000

A = np.zeros((2*N, 2*N))

# Montagem da matriz
for i, (n1, n2) in enumerate(barras):

dx = nos[n2][0] - nos[n1][0]
dy = nos[n2][1] - nos[n1][1]

L = np.sqrt(dx**2 + dy**2)

c = dx / L
s = dy / L

A[2*n1, i] = c
A[2*n1 + 1, i] = s

A[2*n2, i] = -c
A[2*n2 + 1, i] = -s

# Reações
A[0, B] = 1
A[1, B+1] = 1
A[3, B+2] = 1

# Resolve sistema
solucao = gauss_elimination(A, F)

forcas = solucao[:B]
reacoes = solucao[B:]

print("Forças nas barras:")
print(forcas)

print("\nReações de apoio:")
print(reacoes)

# Gráficos
fig, axs = plt.subplots(2,2, figsize=(12,10))

# Estrutura original
for n1, n2 in barras:
axs[0,0].plot(
[nos[n1][0], nos[n2][0]],
[nos[n1][1], nos[n2][1]], 'k-o'
)

axs[0,0].set_title("Treliza Original")

# Mapa de forças
max_f = max(abs(forcas))

for i, (n1, n2) in enumerate(barras): cor =
'blue' if forcas[i] > 0 else 'red' largura = 1
+ 4 * abs(forcas[i]) / max_f

axs[0,1].plot(

```

```

        [nos[n1][0],      nos[n2][0]],
[nos[n1][1],      nos[n2][1]],
color=cor,
linewidth=largura
)

axs[0,1].set_title("Tração e Compressão")

# Barras
axs[1,0].bar(range(B), forcas)

axs[1,0].set_title("Forças por Barra")

# Histograma
axs[1,1].hist(forcas, bins=5)

axs[1,1].set_title("Distribuição das Forças")

plt.tight_layout()
plt.show()

if __name__ == "__main__":
    resolver_treliga()

```

Conclusão:

A versão refeita apresenta maior estabilidade numérica, melhor organização estrutural e gráficos mais coerentes para análise das forças internas da treliça. O método de Gauss continua implementado manualmente, atendendo aos requisitos do problema.